

```
while((buffer = skb_dequeue(&buffer->receive_queue)) != NULL)
{
    nlh = (struct nlmsghdr *)buffer->data;

    //access the data through
    //NLMSG_DATA(nlh)
}
}
```

receive_func function is called in the *sendmsg* system call context. If the task that is to be done with the received message is small, then it can be done in *receive_func*; but if it is not small, then it can block other system calls and can cause delays in the application. So to avoid this, kernel threads can be later used to process the message. For this purpose, the *skb_rcv_datagram* API can be used as follows:

```
struct sk_buff *buffer;
int error;

buffer = skb_rcv_datagram(nlink, 0, 0, &error);
```

In the above code:
 * *nlink* = struct sock* variable is returned by *netlink_kernel_create*.
 * *buffer* = will be the buffer that will contain the Netlink message when the *skb_rcv_datagram* wakes up.

After this call, the calling thread will block and will have to be woken up through *wake_up_interruptible* in *receive_func* callback as follows:

```
void receive_func(struct sock *buffer, int len)
{
    // this will be wake up the thread
    //which has called skb_rcv_datagram
    wake_up_interruptible(buffer->sleep);
}
```

After the thread has woken up, data can be accessed as follows:

```
struct sk_buff *buffer;
int error;
struct nlmsghdr *nlh;

//here the thread will sleep till the message
//is received, after message is received
//receive_func is called which will wake
//up this thread
buffer = skb_rcv_datagram(nlink, 0, 0, &error);

//access the data through buffer variable
```

```
nlh = (struct nlmsghdr *)buffer->data;

//access the data through NLMSG_DATA macro
//in kernel
printf("Message received %s\n", NLMSG_DATA(nlh));
```

To close the Netlink socket allocated, *sock_release* is called as follows:

```
sock_release(&nlink->socket);
```

...where *nlink* is the struct sock * variable returned by *netlink_kernel_create* api.

For sending unicast and multicast messages from the kernel to the process, the following are the APIs:

```
//unicast message sending from kernel
int netlink_unicast(struct sock *ssk, struct sk_buff *skb,
u32 pid, int nonblock);
```

In the above code:

- *ssk* is the struct sock * returned by *netlink_kernel_create*.
- *skb* is the buffer which holds the message.
- *pid* is the ID or address of the process to which the message is to be sent.
- *Nonblock* is the variable to decide whether to block if the process is not present.

```
//multicast message sending from kernel
int netlink_broadcast(struct sock *ssk, struct sk_buff *skb,
u32 pid, u32 group, int allocation);
```

In the above code:

- *group* is the multicast group to which the message is to be sent. This is similar to the *nl_groups* field in the *sockaddr_nl* structure.
- *allocation* is GFP_ATOMIC if called from the interrupt context or GFP_KERNEL if called from the kernel thread. This is due to the fact that the kernel requires multiple buffer allocations to clone a multicast message.

Let's end with an example of message sending in the kernel, as follows:

```
#define NLINK_MSG_SIZE 1024

//allocate netlink socket
struct sock *nlink = netlink_kernel_create(NETLINK_GENERIC,
receive_func);

//allocate socket buffer for message
struct sk_buff *skb = alloc_skb(NLMSG_SPACE(NLINK_MSG_SIZE),
GFP_KERNEL);

//get the header pointer
nlh = (struct nlmsghdr *) skb->data;
```